面向轻量级AI推理的异构加速器

摘要

随着物联网、人工智能和边缘计算的发展，越来越多智能终端需要在本地完成图像识别、语音交互、车载感知和可穿戴健康监测等实时推理任务。边缘端设备通常受限于功耗、成本、存储和算力条件，既要求系统具备较高推理速度，又要求能够灵活部署多种网络。针对上述问题，设计并实现一套面向轻量级神经网络推理框架的CPU-NPU异构协同加速系统。系统以AMD Artix-7 XC7A100T为主要验证平台，由CPU负责模型解析、流程控制、任务调度和非规则算子处理，由FPGA侧NPU负责卷积、全连接等密集算子的并行计算，从而在保留软件框架灵活性的同时提升端侧核心算子的执行效率。

本文围绕端侧资源受限条件下的轻量化部署链路、重复调度开销、硬件计算吞吐三方面问题展开设计。第一，提出轻量级算子统一映射机制，通过统一算子API、运行时指令生成、和自定义指令集架构，把TinyMaix框架中的核心算子转化为底层可执行指令流，从而在不依赖完整商用NPU编译器链路的条件下完成硬件卸载。第二，设计指令流缓存复用机制，对已进入NPU路径的算子完成张量分块、地址映射、硬件指令生成、同步管理和同尺寸指令流复用，降低重复分块、寻址和下发开销。第三，设计细粒度流水化NPU加速结构，将DMA数据搬运、8×8 INT8脉动阵列计算、结果重排和PPU后处理连续组织，减少数据等待和后处理阻塞。

系统在XC7A100T的VexRiscv+TinyMaix路径上验证。Vivado 2026.1布线后50 MHz满足时序，DSP与BRAM利用率分别为88.33%和99.63%。同一模型与输入下，MNIST与语音关键词识别分别获得11.78倍和9.83倍热缓存加速。CPU为片内VexRiscv RV32IM软核，与NPU使用同一50 MHz时钟。

关键词：轻量级算子统一映射；指令流缓存复用；细粒度流水化NPU；灵活部署

目录

第一部分 作品概述 3
1.1 功能与特性 3
1.2 应用领域 3
1.3 主要技术特点 4
1.4 主要性能指标 4
1.5 主要创新点 5
1.6 设计流程 5
第二部分 系统组成及功能说明 6
2.1 整体介绍 6
2.2 硬件系统介绍 7
2.2.1 硬件整体介绍 7
2.2.2 脉动阵列与卷积分块 7
2.2.3 数据缓存与DMA访存调度 9
2.2.4 PPU后处理与量化输出流水 11
2.3 软件系统介绍 12
2.3.1 软件整体介绍 12
2.3.2 轻量化算子映射与部署 13
2.3.3 运行时调度与指令流缓存 14
2.3.4 软硬件协同执行流程 16
2.4 设计难点与取舍 17
第三部分 完成情况及性能参数 19
3.1 整体介绍 19
3.2 工程成果 20
3.2.1 硬件模块仿真 20
3.2.2 FPGA综合与实现 22
3.2.3 软件运行时成果 23
3.3特性成果 24
3.3.1缓存状态与推理性能 24
3.3.2 端到端推理性能 24
第四部分 总结 25
4.1 可扩展之处 25
4.2 心得体会 25
第五部分 参考文献 26

第一部分 作品概述

1.1 功能与特性

NPU（Neural Processing Unit，神经网络处理单元）是指面向神经网络推理任务设计的专用计算单元。与CPU依靠通用指令逐条执行不同，NPU通常通过并行乘加阵列、片上缓存和数据搬运模块，加速卷积、矩阵乘法和全连接等神经网络中的密集计算。本作品中的NPU由8×8 INT8脉动阵列、后处理单元PPU、片上缓存和DMA等模块组成，主要负责执行规则密集型算子；CPU则负责模型解析、任务调度、复杂控制和不适合硬件加速的补充算子。

针对端侧推理中CPU密集算子计算能力有限、专用加速器适配性不足以及端侧NPU部署链路复杂等问题，本文提出一套面向轻量级神经网络推理框架的CPU-NPU异构协同加速系统。系统面向“电路与嵌入式系统类”，以AMD Artix-7 XC7A100T FPGA为主要验证平台，由CPU负责模型解析、流程控制、任务调度和非规则算子处理，由FPGA侧NPU承担卷积、逐点卷积、全连接等计算密集型任务。同时本作品更加关注端侧轻量化部署链路，重点解决轻量级推理框架如何通过开放、可控的软件后端调用自定义NPU硬件的问题。

1.2 应用领域

随着边缘人工智能的发展，越来越多智能终端需要在终端完成图像识别、语音关键词识别、工业检测、可穿戴健康监测等实时推理任务。这类设备通常受限于功耗、成本、存储和散热条件，既要求系统具备较低功耗，又要求在有限资源下完成一定强度的神经网络计算。与此同时，不同应用采用的神经网络模型在算子类型、输入尺寸和计算规模上存在明显差异。例如图像识别任务中卷积和逐点卷积占比较高，语音唤醒任务中可能包含一维卷积或全连接运算，目标检测和工业缺陷检测则对吞吐率和实时性提出更高要求。因此，边缘端推理加速不仅需要提升核心算子的计算效率，还需要具备对多类模型和不同输入尺寸的适配能力。

现有轻量级推理框架虽然具有较好的模型部署灵活性，但在端侧设备上大多依赖CPU完成卷积、全连接等密集乘加计算。CPU具备较强的通用控制能力，适合完成模型加载、流程调度和分支控制，但受限于并行计算能力和访存带宽，在复杂AI推理任务中容易出现算力不足、能效不高的问题。固定数据流的专用加速

器虽然执行效率较高，但通常面向特定模型结构进行优化，难以适应边缘AI场景中模型结构多变、输入尺寸不统一的需求。因此，CPU-NPU异构架构成为边缘端推理加速的重要方向：CPU负责模型解析、流程控制和任务调度，NPU负责卷积、全连接等密集算子的并行计算[1]。要让这种异构系统真正大规模运用到终端取得效果，还需要解决硬件灵活部署和推理提速两个关键的问题。



图1-1边缘智能应用场景示意图

1.3 主要技术特点

围绕这一目标，作品从轻量化部署、软硬件交互开销和硬件计算吞吐三个层面展开设计：首先提出轻量化算子映射与异构协同执行机制，在不依赖完整离线编译工具链的条件下，使规则密集型算子由NPU加速、非密集或暂未支持的算子继续由CPU执行。系统在VexRiscv+TinyMaix路径上完成验证，说明作品不是面向单一网络的固化电路，而是面向轻量级推理框架的端侧异构协同加速平台。其次设计运行时调度与轻量级指令流缓存机制，将不同尺寸张量映射到底层固定规模阵列，并在同尺寸重复推理时复用已生成的硬件指令； 最后我们设计了细粒度流水化NPU加速结构，将DMA数据搬运、脉动阵列计算、结果重排和PPU后处理进行流水化组织，提高乘加阵列与后处理通路的整体吞吐率。

1.4 主要性能指标

布线后50 MHz时序报告中，WNS为2.96 ns，WHS为0.036 ns，setup与hold均满足约束。Vivado估算片上总功耗为0.429 W，其中动态功耗为0.325 W，静态功

耗为0.104 W。估算采用布线后设计、默认开关活动和25°C默认散热模型，未导入SAIF活动文件；该数值为FPGA片上估算功耗，不包括开发板外设和电源损耗。

MNIST与语音关键词识别在最终XC7A100T演示平台上分别取得11.78倍、9.83倍热缓存加速。

1.5 主要创新点

本文技术路线围绕三项创新点展开。

其一是NPU算子映射与轻量化一键部署机制：作品在不依赖完整离线编译工具链的条件下，在运行时完成算子参数解析、数据分块和指令生成，使规则密集型算子由FPGA侧NPU加速执行，非密集或暂未支持的算子仍由CPU侧执行，提升系统对多类模型和不同输入尺寸的适配能力。

其二是算子运行时调度与轻量级指令流缓存机制：运行时负责将不同尺寸的张量计算映射到底层固定规模阵列，并生成对应硬件指令；指令流缓存机制在首次推理时保存指令序列，后续同尺寸推理可直接复用，从而降低重复指令生成和CPU调度开销。

其三是细粒度流水化NPU加速结构：作品将DMA数据搬运、8×8 INT8脉动阵列计算、结果重排和PPU后处理进行流水化组织，通过权重缓存、激活FIFO和访存调度减少模块间等待，提高乘加阵列和后处理通路的整体吞吐率。

1.6 设计流程

作品围绕轻量化部署机制、算子运行时调度与指令流缓存机制、细粒度流水化NPU加速结构，形成从问题分析、方案论证、软件运行时、硬件流水到系统测试的完整技术闭环。硬件侧通过模块仿真、资源和时序结果验证；运行时侧通过缓存状态对比测试验证。

第二部分 系统组成及功能说明

2.1 整体介绍

硬件加速器内部的系统架构如图2-1所示。该架构主要包含全局控制单元、脉动阵列单元、后处理单元、数据搬运CDMA引擎、全局缓存，以及负责连通各功能模块的AXI Crossbar片上互连矩阵。数据搬运与片上存储网络主要由CDMA引擎、AXI互连矩阵与全局缓存构成。 CDMA引擎作为数据总线的主设备，负责执行片上共享BRAM与片上全局缓存之间 的数据预取与结果写回。全局控制单元包含控制与状态寄存器（CSR）、指令队列以及指令预取DMA。其内部的控制器负责对主机下发的指令进行译码，根据指令类型将具体的控制与计算任务分发至各子模块的局部指令队列。脉动阵列单元内部包含局部指令队列与控制器、本地数据缓冲、专用的读DMA以及8 × 8的乘加计算阵列。后处理单元同样包含独立的指令队列与局部控制器，其计算流水线由偏置相加、量化重缩放及ReLU激活逻辑组成，并配备输出结果收集模块与用于数据写回的写DMA。

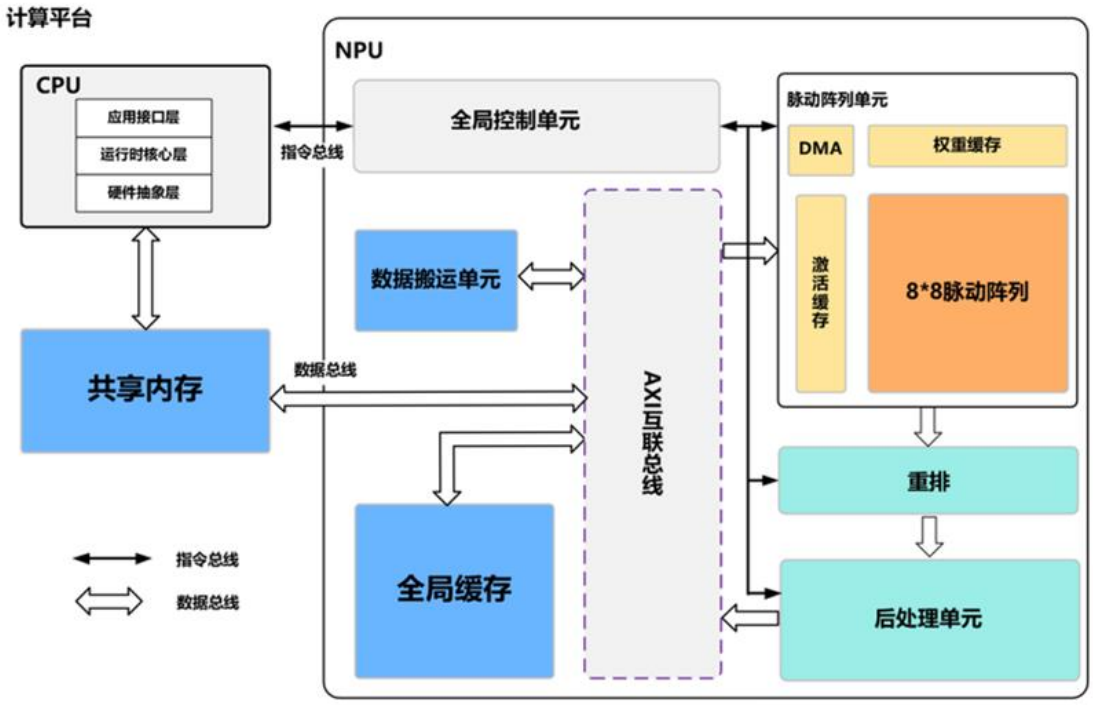


图2-1硬件加速器系统架构框图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

轻量级神经网络推理虽然包含激活、池化、归一化等多类操作，但主要计算量和访存开销通常集中在规则乘加算子上。标准卷积在空间窗口与卷积核之间反复执行乘加，逐点卷积可转化为通道维度上的矩阵乘法，全连接层天然对应矩阵向量或矩阵矩阵乘法。这些算子具有计算规则、数据访问可预测、复用关系明确的特点，适合由8×8 INT8脉动阵列批量处理。围绕这一特点，本作品没有只放置一个孤立乘加阵列，而是采用细粒度流水化NPU加速结构，将共享内存到片上缓存的数据搬运、权重和激活值加载、阵列乘加、输出重排、PPU后处理和结果写回按阶段组织起来，使各阶段尽量并行推进。这样可以减少“搬运完成后阵列才启动”或“阵列输出后CPU再做后处理”的停顿[5][6]，避免硬件加速收益被访存等待和后处理开销抵消。

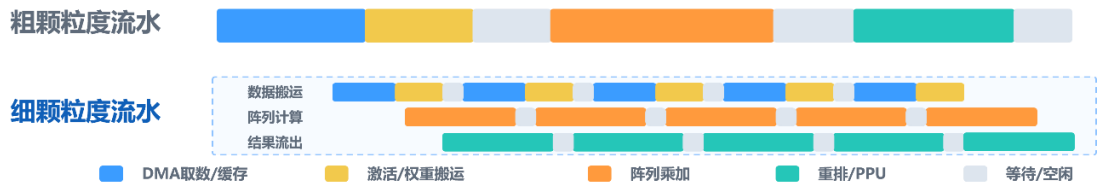


图2-2细粒度流水结构示意图

2.2.2 脉动阵列与卷积分块

作品采用8×8 INT8脉动阵列作为核心计算结构，每个PE单元执行定点乘加，并按固定方向传递权重、激活值或中间结果。脉动阵列与张量专用数据通路在深度学习加速器中应用广泛，相关研究表明其优势来自规则数据复用和高并行乘加吞吐[6][12]。与CPU逐指令串行执行不同，脉动阵列依靠数据流驱动，使多个PE在同一周期并行工作，适合矩阵乘法和卷积展开后的规则乘加[10][11]。

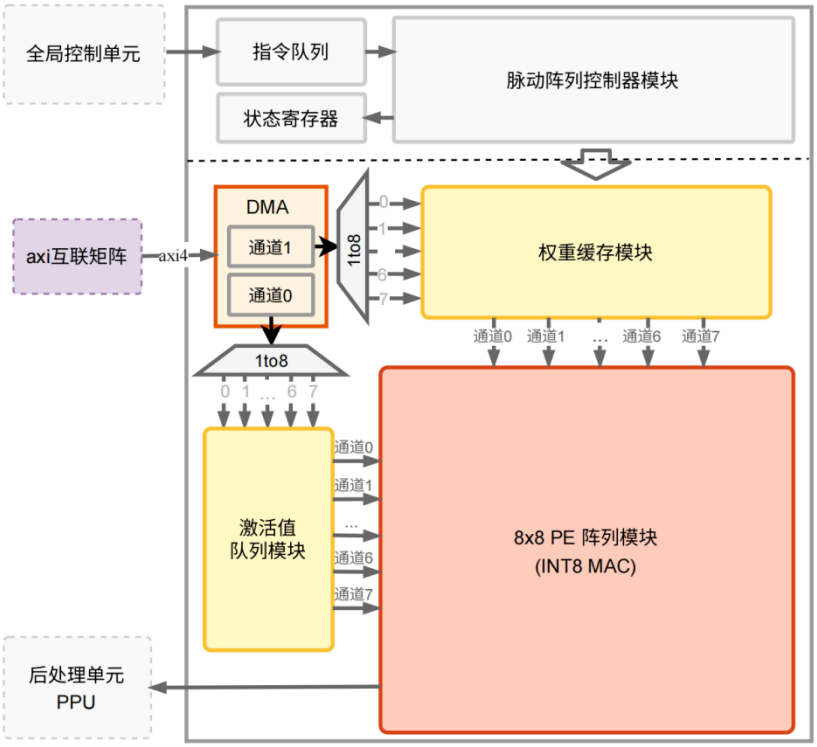


图2-3 8×8脉动阵列数据流结构

卷积映射到脉动阵列时，需要同时考虑输入特征图分块、卷积核通道分块和输出通道分块。图2-4展示了卷积计算分块思路：输入通道被拆分为多个子块，权重按输出通道组织，阵列在不同时间步完成局部结果累加。该方式能够在有限阵列规模下覆盖更大的网络层尺寸，但也要求运行时准确处理尾块、地址步进和最后一次累加标志，使阵列输出能够与后续PPU流水自然衔接。

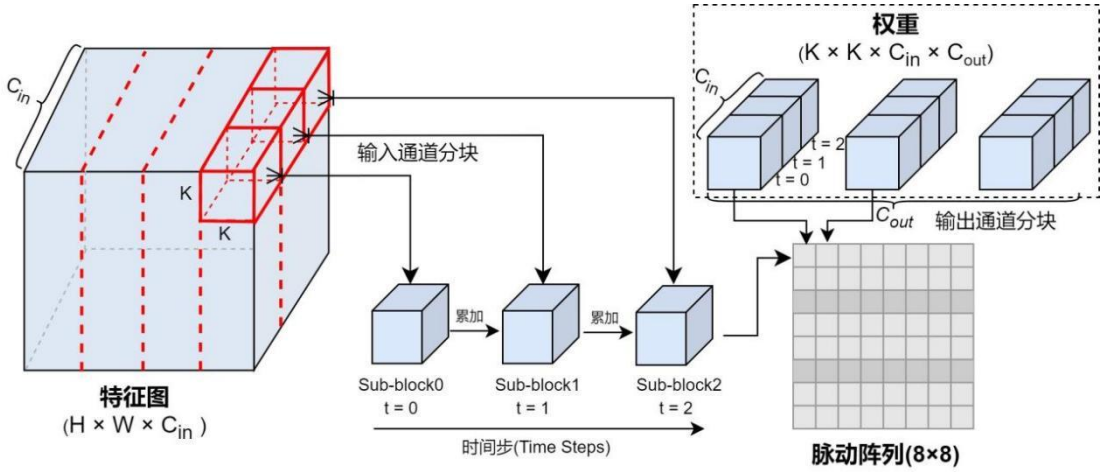


图2-4卷积计算分块与阵列映射

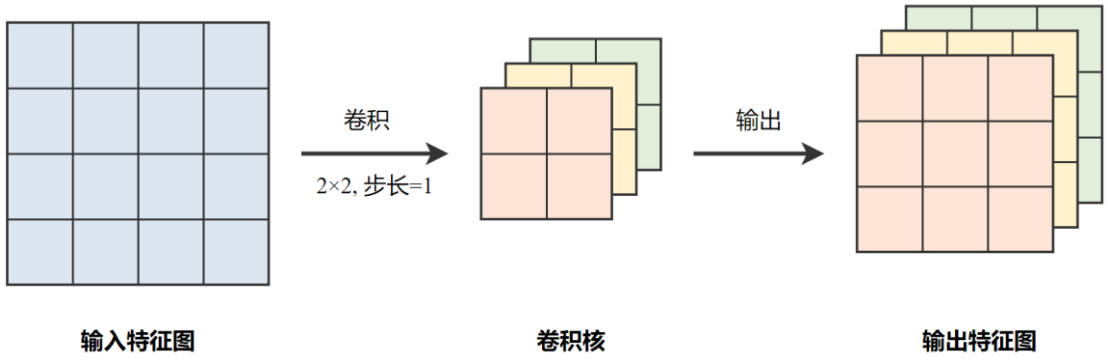


图2-5标准卷积算子计算过程

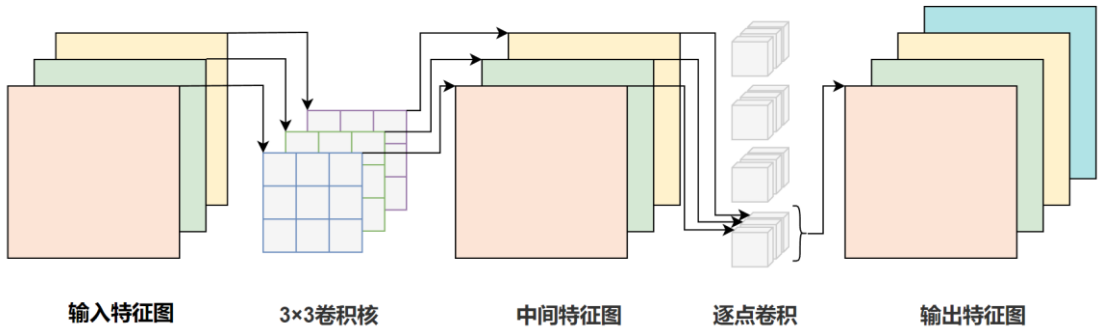


图2-6深度可分离卷积算子结构

细粒度流水化结构还需要兼顾不同算子的计算形态。深度可分离卷积中的深度卷积按通道独立计算，权重复用和跨通道累加较弱，更容易受访存带宽、片上缓存容量和尾块调度影响；逐点卷积虽然卷积核尺寸为1×1，但通道维度乘加密集，通常更适合脉动阵列发挥吞吐优势。本作品还分析了INT8量化与后处理机制：阵列内部使用INT32保存累加结果，输出必须经过偏置相加、ReLU激活、量化重缩放和饱和截断后才能进入下一层。若这些步骤脱离硬件流水，会导致中间结果反复写回共享内存并由CPU重新遍历[7]。为此，NPU在阵列之后设置PPU后处理单元，并配合输出收集链和写回DMA，使硬件路径覆盖从数据搬运、阵列计算到量化输出的完整密集算子流程。该设计的重点不是追求单个PE的峰值性能，而是在端侧FPGA资源范围内提高阵列、访存和后处理通路的整体有效吞吐。

2.2.3 数据缓存与DMA访存调度

仅有计算阵列并不能保证系统加速，访存调度同样关键。系统在硬件侧设置权重缓存、激活FIFO、全局buffer和DMA通道，使数据能够按照阵列消费节奏进入计算核心。权重缓存采用乒乓结构，一组buffer被阵列读取时，另一组buffer准备下一轮数据；激活FIFO用于吸收输入数据到达节奏和阵列计算节奏之间的差

异。DMA、缓存与FIFO的配合使硬件不再按“搬完一批再算一批”的粗粒度方式运行，而是尽量让搬运、计算和写回形成连续流水[13]。

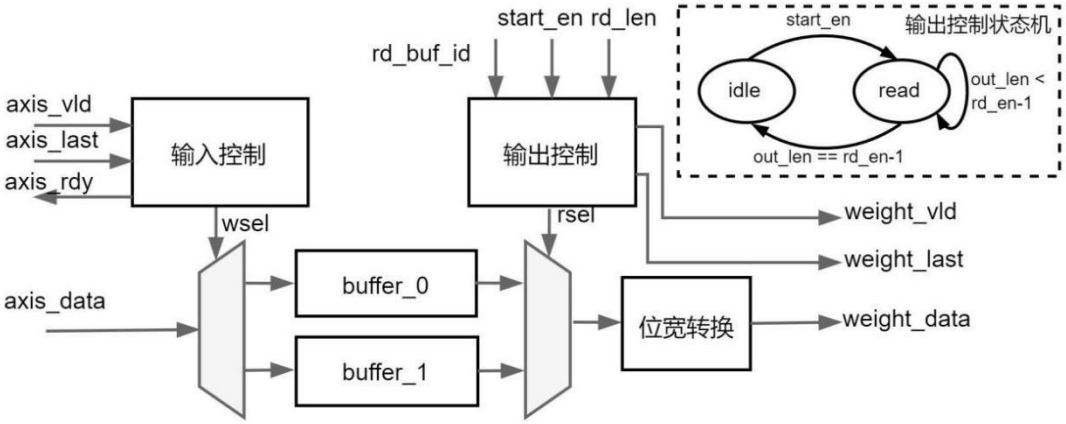


图2-7权重缓存与输出控制结构

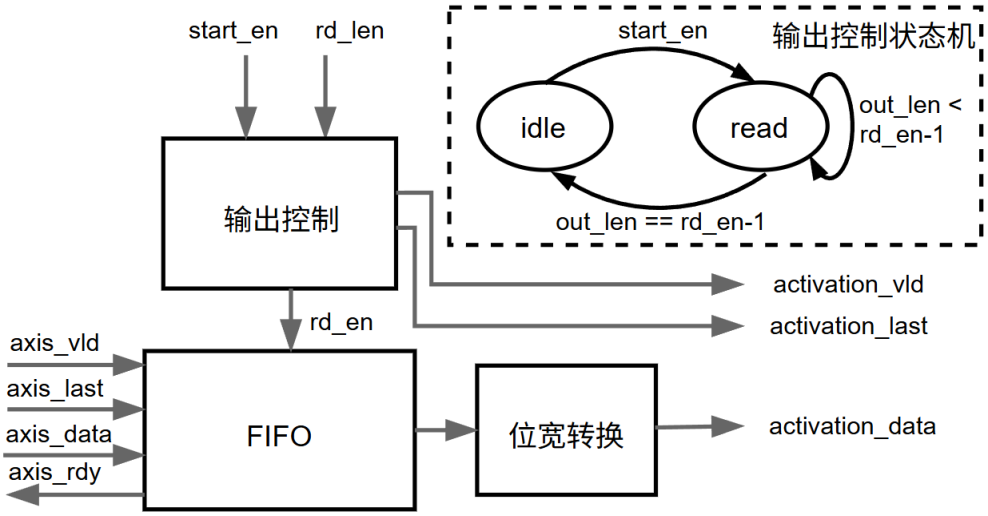


图2-8激活FIFO与输出状态控制

全局控制单元将指令队列、状态寄存器、同步模块和各子模块控制信号连接起来，是硬件系统的调度中心。它负责协调DMA、脉动阵列和PPU的执行顺序，在局部模块完成后更新同步状态，并在全局任务完成后通过中断通知CPU。这种控制方式既保证流水中各阶段的数据依赖正确，也为轻量级指令流的批量预取和复用提供硬件入口。

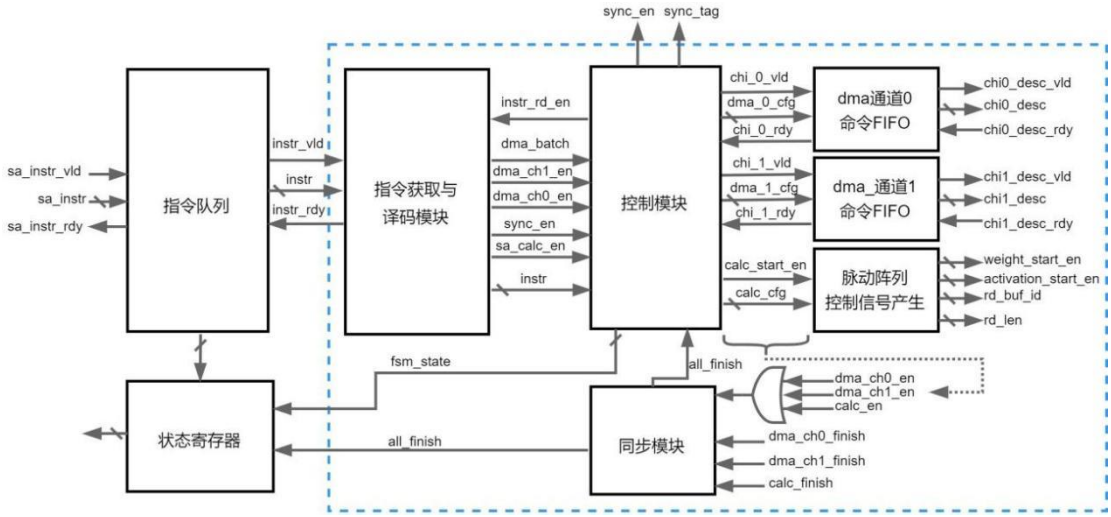


图2-9全局控制单元结构

2.2.4 PPU后处理与量化输出流水

轻量级推理通常采用INT8量化，因此硬件输出不能只停留在高位宽累加结果。整数量化推理通常需要在累加后完成尺度因子、零点偏移和饱和截断等处理[10]。PPU后处理单元接在脉动阵列输出之后，依次完成偏置相加、ReLU激活、量化重缩放、零点偏移和饱和截断，使输出数据可以直接写回为框架可继续使用的量化张量。

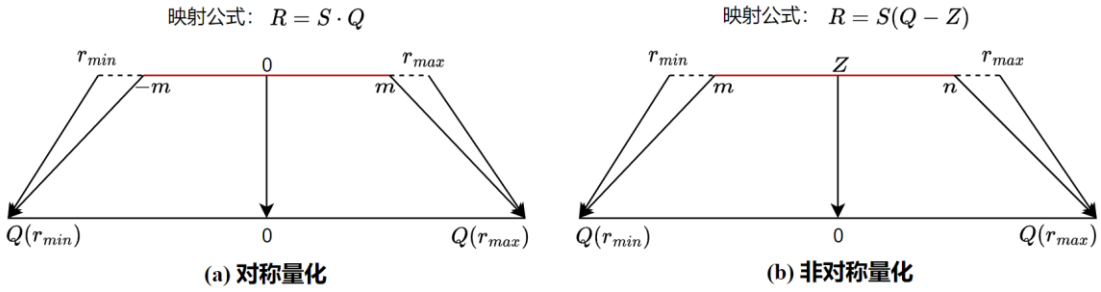


图2-10对称量化与非对称量化映射关系

将PPU后处理纳入硬件流水，可以减少中间结果在CPU与FPGA之间的往返，也避免CPU重新遍历输出特征图执行激活和缩放。对于卷积层、逐点卷积层和全连接层占比较高的网络，细粒度流水化NPU加速结构覆盖的不只是乘加阶段，而是从数据搬运、阵列计算到后处理写回的完整计算密集型算子路径[14]。

2.3 软件系统介绍

2.3.1 软件整体介绍

软件设计部分主要承接后两项机制：算子运行时调度与轻量级指令流缓存机制、轻量化算子映射与异构协同执行机制。其中，运行时调度解决已选硬件算子的分块、寻址、指令生成和同步管理问题。若上层推理框架直接面对寄存器地址、DMA描述符和中断状态，软件会与具体硬件实现强绑定[15]，后续更换主控CPU、调整加速器地址空间或扩展硬件模块时都需要大范围修改。因此，作品在软件侧划分应用接口层、运行时核心层和硬件抽象层，让运行时专门负责已选硬件算子的张量分块、地址映射、指令生成、指令缓存和同步管理。

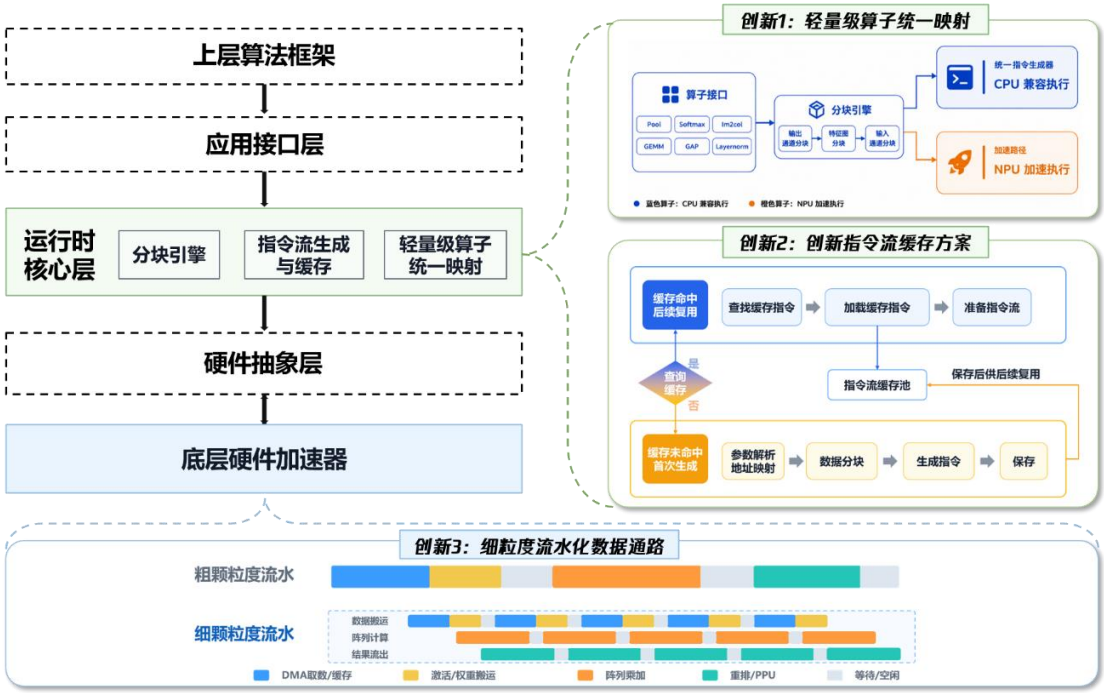


图2-11算子驱动及运行时分层架构

应用接口层接收上层传入的卷积、全连接、矩阵乘法等硬件算子调用。运行时核心层解析张量维度，进行输入、权重和输出的分块，计算共享内存地址，并生成硬件机器码。对于网络结构固定、输入尺寸固定或重复推理的任务，第一次调用时生成的指令序列会被缓存，后续推理直接复用。硬件抽象层封装寄存器读写、指令DMA批量下发和中断等待，使上层代码不需要感知底层总线细节。这一部分对应的是调度效率和重复控制开销问题。

2.3.2 轻量化算子映射与部署

本节要解决的问题是端侧轻量化部署问题：如何在不依赖完整离线编译工具链的条件下，使轻量级推理框架中的模型能够快速接入自研NPU，并在运行时完成算子映射、指令生成和硬件协同执行。



图2-12模型直连式轻量化部署机制

本文将模型部署链路划分为“神经网络、算子接口、分块引擎、统一指令生成器和硬件加速器”五个层次。最上层是待部署的神经网络模型。不同模型虽然网络结构不同，但在推理执行过程中最终都会分解为若干基础算子，例如卷积、池化、Softmax、im2col、GEMM、GAP和LayerNorm等。因此，本文不针对某一个固定网络设计专用数据流，而是在模型和硬件之间保留统一的算子接口，使不同模型能够通过相同的软件入口接入底层加速器。

在算子接口下方，分块引擎负责根据算子类型、张量形状、8×8阵列规模和片上缓存容量，将大规模张量计算切分为硬件能够执行的子任务。对于卷积和全连接等密集计算，系统会将其规整为矩阵乘加任务，并按照输出通道、特征图空间位置和输入通道维度进行分块；对于暂未支持或硬件利用率较低的算子，则保留在CPU或软件框架中执行。这样既能让规则密集型算子进入硬件路径，又不会为了覆盖少量非规则算子引入复杂硬件和重型编译器。

分块结果随后进入统一指令生成器。该模块根据每个子任务的数据地址、搬运长度、计算规模和后处理参数，生成硬件可解析的64/128-bit自定义指令，包括数据搬运指令、脉动阵列计算指令、PPU后处理指令和同步指令。硬件抽象层进一步负责寄存器访问、指令DMA批量下发、中断等待和状态同步。通过这种方式，上层推理框架或用户应用不需要直接面对底层寄存器、DMA描述符和同步信号，底层硬件也不需要针对每个模型重新设计控制逻辑。

与传统商用NPU“先离线编译，再部署执行”的方式相比，本文采用运行时直接接入模型的方式。传统部署方案通常需要先将模型导入专用工具，经过离线优化、算子划分、NPU专用执行描述生成，再重新编译并下载到目标设备。当模型结构、输入尺寸或部署任务发生变化时，往往需要重新执行部分离线流程，部署链路较重。而本文保留推理框架入口，在运行时完成算子识别、张量分块、地址映射和硬件指令生成，使模型接入后即可进入软硬件协同执行流程.

该机制的优势在于轻量化部署。对于模型结构和输入尺寸稳定的重复推理任务，首次生成的硬件指令可以缓存并复用，避免重复执行分块、寻址和指令生成；对于模型变化或多模型部署场景，系统可以通过统一算子接口和运行时分块机制重新完成映射，而不必重走完整商用NPU离线编译部署链路。因此，本设计更适合端侧轻量模型快速部署、多模型适配等场景。

TinyMaix面向MCU/TinyML场景，框架较轻[14]，通常搭配VexRiscv软核进行验证。

在这一设计中，CPU与NPU的边界保持清晰：CPU负责模型解析、算子选择、非规则计算和全局控制，NPU负责卷积、逐点卷积、全连接等规则密集型计算。统一硬件算子接口向上屏蔽推理框架的调用差异，向下保持对同一套NPU硬件通路的访问方式。这样既能复用原有轻量级框架生态，又能在不构建完整离线编译工具链的条件下完成端到端推理加速[17]。

2.3.3 运行时调度与指令流缓存

算子运行时调度解决的是“已进入硬件路径的算子如何被高效执行”的问题。运行时核心层接收上层传入的算子尺寸、权重地址、输入输出张量地址和量化参数后，首先根据底层8×8阵列规模、片上缓存容量和输出通道数量进行张量分块；

随后计算每个子块在共享内存和片上缓存中的地址，确定权重、激活值、输出特征图和量化参数的访问顺序；最后将这些分块任务翻译为硬件可解析的搬运指令、计算指令、后处理指令和同步指令。硬件抽象层进一步封装寄存器访问、指令下发、DMA调度和中断同步。这个层次只关注硬件任务的组织、缓存与下发，算子执行路径的选择放在前文的轻量化算子映射机制中实现。

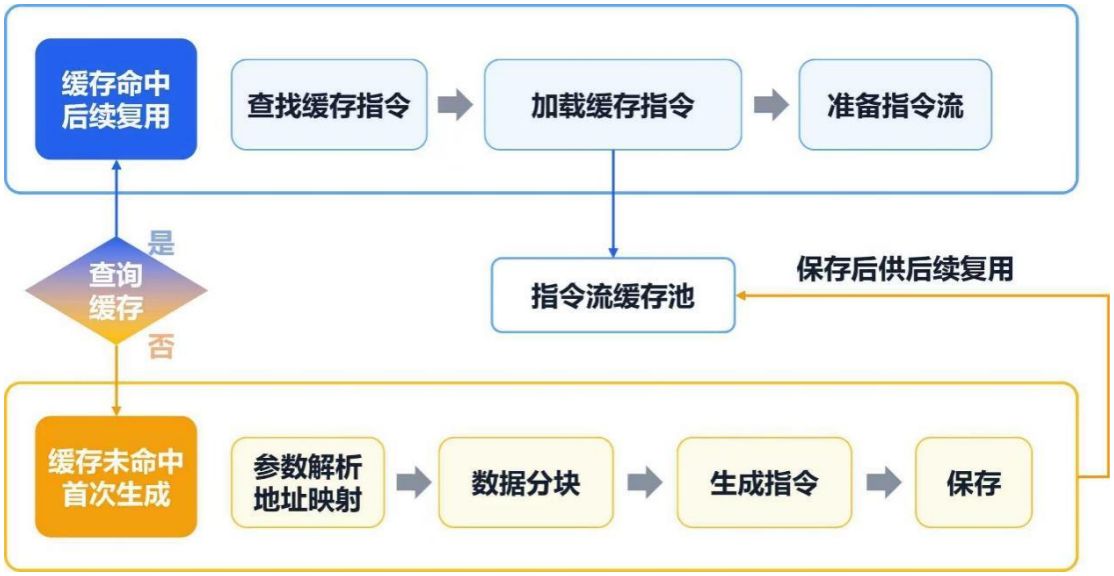


图2-13运行时调度指令流缓存机制

该图展示了本作品运行时调度核心层中的指令流缓存机制。系统在推理开始时首先查询缓存：若缓存命中，则直接加载已保存的缓存指令，快速完成指令流准备并进入后续执行阶段；若缓存未命中，则进入首次生成路径，依次完成参数解析与地址映射、数据分块、底层硬件指令生成，并将生成结果保存至缓存池中，供后续重复推理直接复用。通过这一机制，系统避免了重复推理过程中频繁进行参数解析、数据分块和指令生成，从而显著降低了CPU调度开销与软硬件交互成本，提高了整体推理执行效率。

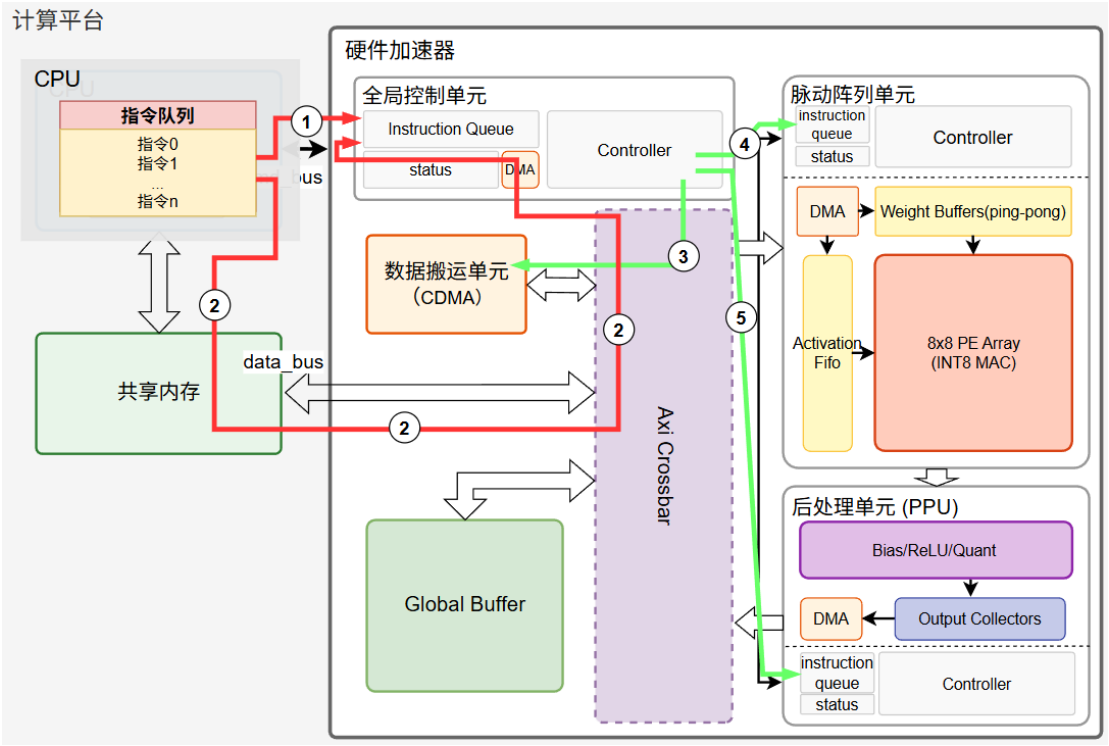


图2-14运行时调度 指令生成与缓存复用流程

图2-14展示了运行时调度到硬件执行的流程。对于一个已经确定由NPU执行的算子，运行时先沿输出通道、空间像素和输入通道展开维度进行分块，再为每个子任务生成数据搬运、阵列计算、PPU后处理和同步指令。首次推理时，运行时需要完整执行分块计算、地址规划和64/128-bit指令生成；当网络结构和输入尺寸保持一致时，生成的指令流可以被缓存并在后续推理中复用。缓存命中后，系统只需重新绑定输入输出数据地址或加载已保存的指令块，避免重复执行多层循环切块和逐条寄存器配置。硬件侧的指令DMA可以从共享内存批量预取指令，局部同步和全局同步指令用于保证DMA、脉动阵列和PPU的依赖顺序。由此，轻量级指令流缓存机制把CPU的工作重点从细粒度控制转为批量任务提交，降低重复推理场景下的软件调度开销。

2.3.4 软硬件协同执行流程

本节关注的是模型接入后的运行时映射与协同执行。一次推理任务开始后，CPU侧推理框架仍负责模型加载、计算图遍历和普通软件算子执行；当框架遇到硬件支持的Conv、DWConv、PWConv、FC等算子时，应用接口层将框架参数封装为统一硬件算子描述，运行时核心层进一步完成张量分块、物理地址计算、量

化参数组织和自定义机器码生成，最终由硬件抽象层通过寄存器、指令DMA和中断机制启动NPU执行。

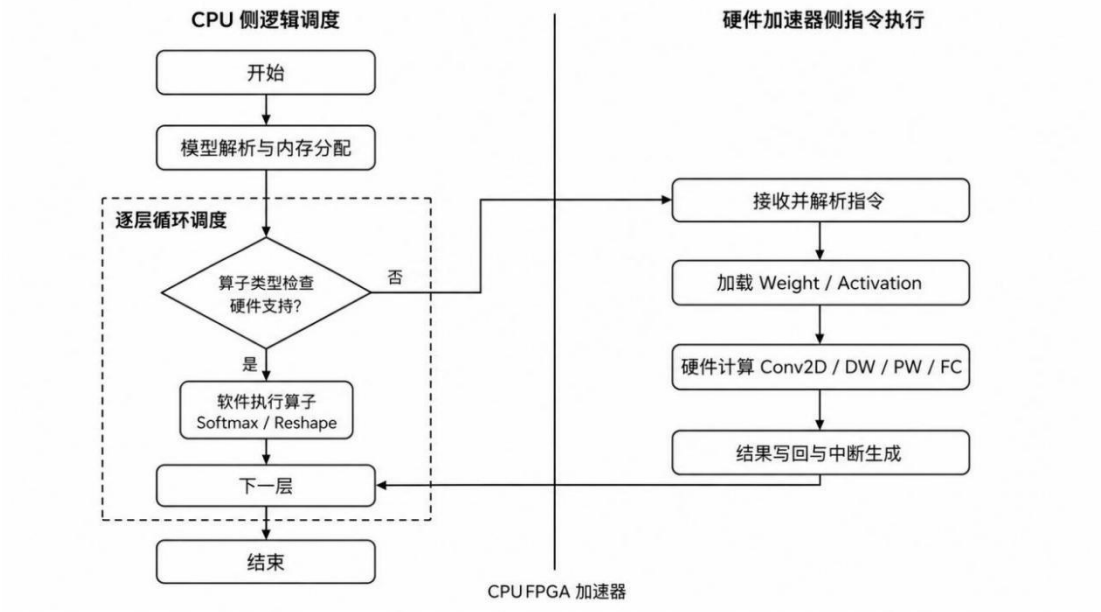


图2-15软硬件协同计算流程

与传统“离线编译后再部署”的方式相比，该机制将部署压力从完整离线工具链转移到轻量运行时中。系统不需要为每次模型调整重新生成完整工程或重新下载整套部署结果，而是保留推理框架入口，在运行时根据当前算子参数生成或复用硬件指令。这样既能保持轻量级框架的灵活性，又能将密集乘加算子下沉到NPU执行，适合端侧模型快速迭代和多模型部署。

2.4 设计难点与取舍

第一，轻量化部署与多模型适配难。边缘端轻量级推理任务通常运行在资源受限的嵌入式平台上，不适合引入复杂模型编译器或完整离线编译工具链。复杂编译器虽然可以进行深度图优化和算子融合，但部署成本高、适配链路长，也不利于轻量模型在端侧快速验证和运行[4]。本作品需要在运行时完成算子参数解析、张量分块、地址映射和指令生成，并根据算子类型决定由NPU加速或由CPU继续执行，从而兼顾TinyMaix框架的部署灵活性、不同输入尺寸的适配能力和端到端推理效率。

第二，运行时调度开销大。在CPU-NPU异构系统中，推理任务并不是简单地把一个算子交给硬件执行。CPU需要完成数据分块、地址规划、共享内存管理、

指令生成和同步等待等工作，如果每次推理都重复生成和下发大量控制指令，数据搬运和控制开销会削弱甚至抵消NPU的计算收益。因此，如何在推理框架和底层硬件之间形成低耦合、低开销的算子运行时调度链路，并通过指令流缓存减少同尺寸重复推理中的指令生成和CPU调度开销[3]，是作品实现的第二项难点。

第三，持续吞吐能力不足。端侧平台资源有限，卷积、全连接等密集算子虽然计算形式规则，但在实际执行中不仅受乘加阵列规模限制，还容易受到数据搬运、阵列计算、结果重排和后处理等环节的共同制约。如果各模块之间衔接不紧密，就会出现计算阵列等待数据、结果写回阻塞、后处理跟不上等问题，导致硬件峰值算力难以持续发挥。针对这一问题，本作品提出细粒度流水化数据通路，在XC7A100T有限的DSP、BRAM和布线资源下，将8×8 INT8脉动阵列、权重缓存、激活FIFO、DMA访存调度和PPU后处理进行协同设计，使数据搬运、阵列计算、结果重排和后处理形成连续流水，减少中间等待与模块堵塞，从而提高整体吞吐率。

第三部分 完成情况及性能参数

3.1 整体介绍

本文面向轻量级神经网络推理框架设计并实现了一套CPU-NPU异构硬件加速系统。作品围绕轻量化部署机制、算子运行时调度与指令流缓存机制、细粒度流水化NPU加速结构，形成了从问题分析、方案论证、软件运行时、硬件流水到系统测试的完整技术闭环。

系统演示由运行上位机的电脑和XC7A100T FPGA开发板组成。电脑负责输入与结果展示，开发板执行TinyMaix推理，通过串口返回识别结果与CPU/NPU计时数据。图3-1（a）为现场演示实物，图3-1（b）为上位机界面。

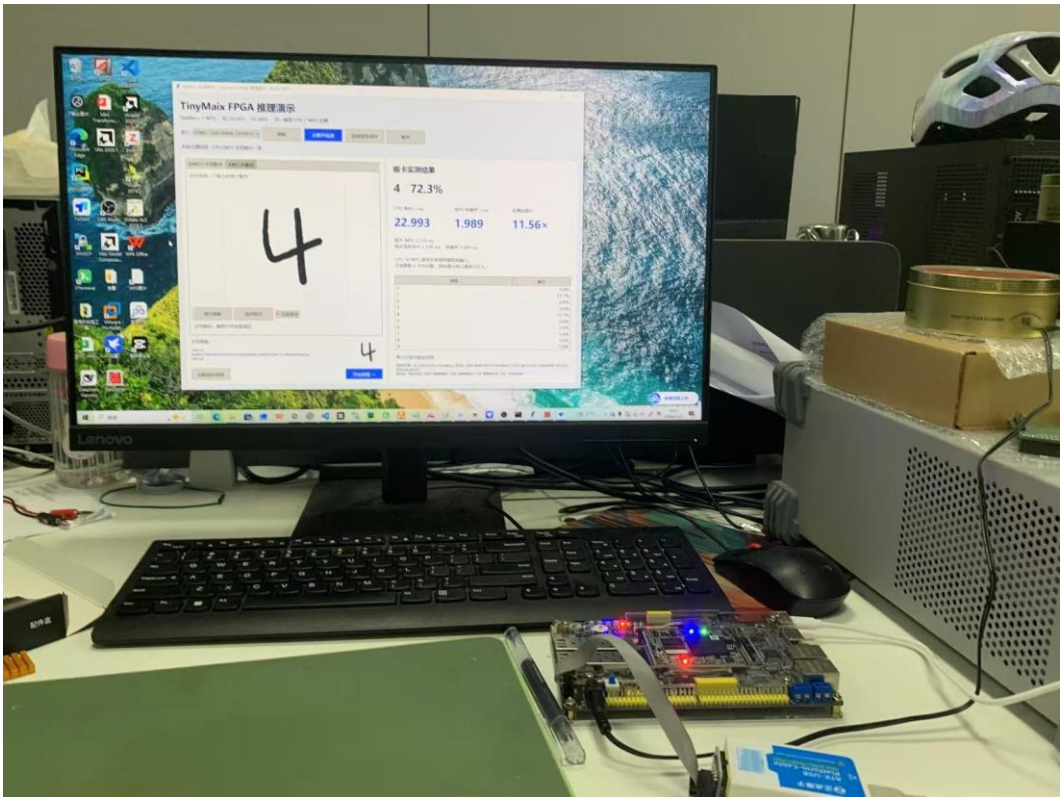


图3-1（a）系统现场演示实物

照片为MNIST手写输入的一次现场演示，界面中的识别结果、耗时与加速比对应图中输入；固定测试样本的基准性能见3.3节。



图3-1（b）XC7A100T上位机板级推理验证

3.2 工程成果

3.2.1 硬件模块仿真

系统首先通过模块级仿真验证关键子模块的功能正确性，并把验证对象对应到前文三项机制。对于细粒度流水化NPU加速结构，测试平台构造固定矩阵乘法和随机矩阵乘法用例，验证8×8 INT8脉动阵列在不同输入激励下的乘加结果；同时验证PPU的旁路模式和全链路模式，检查偏置、ReLU、重缩放和饱和截断在边界数据下是否正确。对于算子运行时调度与轻量级指令流缓存机制，指令DMA测试预先在共享内存写入连续指令序列，再由硬件批量搬入指令队列，用以验证批量指令加载过程中的顺序、丢失和重复问题。

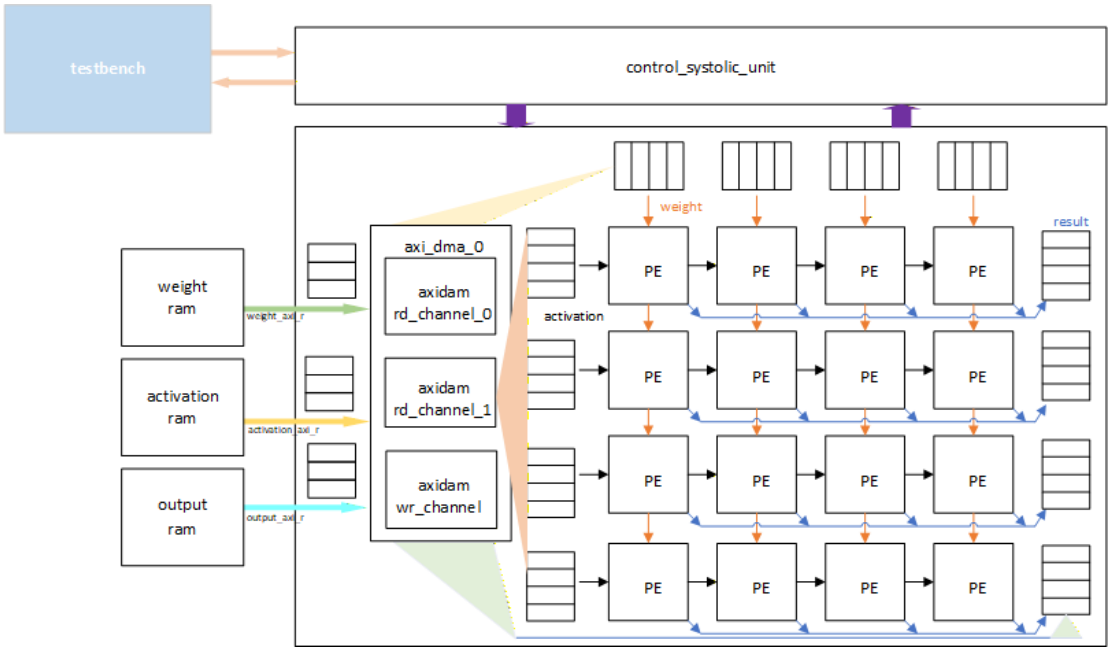


图3-2 脉动阵列模块仿真测试平台

```
[======= times: 4998 =======
matrix a : 17, 12
matrix b : 12, 10
result: pass
[======= times: 4999 =======
matrix a : 29, 10
matrix b : 10, 20
result: pass
[======= times: 5000 =======
matrix a : 20, 27
matrix b : 27, 17
result: pass
[============= summary ===============]
total   times : 5000
passed  times : 5000
failed  times : 0
timeout times : 0
```

图3-3 随机矩阵回归测试结果

仿真结果显示，指令DMA能够正确完成批量指令加载，为轻量级指令流缓存后的批量下发提供硬件支撑；脉动阵列在随机矩阵用例中完成多轮回归测试，硬件结果与参考模型一致，说明基础乘加路径可靠；后处理单元在旁路模式和全链路模式下均能得到符合预期的输出，说明PPU可以接入阵列输出流水。上述仿真主要验证了细粒度流水化NPU加速结构的功能正确性，同时验证了运行时指令流能够被硬件侧稳定接收和调度。

3.2.2 FPGA综合与实现

作品采用Vivado 2026.1，在XC7A100T（xc7a100tfgg484-2）上完成综合、布局布线和位流生成，CPU与NPU工作频率均为50 MHz。整套系统资源占用为LUT 41174/63400（64.94%）、FF 38150/126800（30.09%）、BRAM 134.5/135（99.63%）、DSP 212/240（88.33%）。统计范围包含VexRiscv、NPU、片上存储和互连。DSP用于阵列及相关乘加和后处理逻辑，BRAM用于程序、数据、共享存储及NPU缓存。BRAM利用率接近器件上限。

Resource	Utilization	Available	Utilization %
LUT	41174	63400	64.94
LUTRAM	10108	19000	53.20
FF	38150	126800	30.09
BRAM	134.50	135	99.63
DSP	212	240	88.33
IO	4	285	1.40

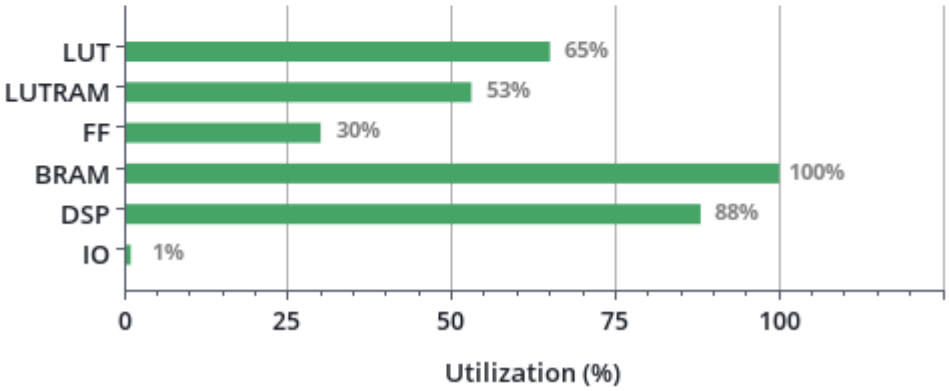


图3-4 FPGA资源利用率统计表

XC7A100T整套系统资源利用率

LUT	FF	BRAM	DSP
64.94%	30.09%	99.63%	88.33%

图3-5 FPGA资源利用率对比

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	2.960 ns	Worst Hold Slack (WHS)	0.036 ns	Worst Pulse Width Slack (WPWS)	8.870 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	197548	Total Number of Endpoints	197548	Total Number of Endpoints	51031

All user specified timing constraints are met.

图3-6时序收敛摘要

布线后50 MHz时序报告中，WNS为2.96 ns，WHS为0.036 ns，setup与hold均满足约束。Vivado估算片上总功耗为0.429 W，其中动态功耗为0.325 W，静态功耗为0.104 W。估算采用布线后设计、默认开关活动和25°C默认散热模型，未导入SAIF活动文件；该数值为FPGA片上估算功耗，不包括开发板外设和电源损耗。

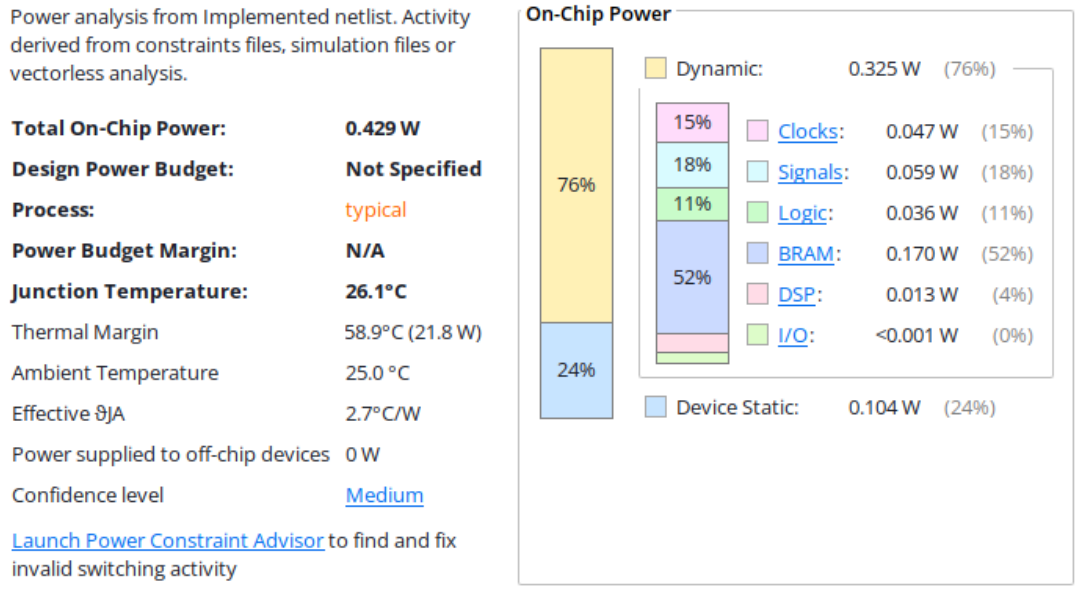


图3-7 XC7A100T片上功耗估算

3.2.3 软件运行时成果

与只针对单一网络的硬连线加速器相比，本作品更加关注三条技术链路的边界清晰与测试闭环。硬件侧通过DMA、脉动阵列、结果重排和PPU后处理流水提供细粒度流水化NPU加速结构，并通过模块仿真、资源和时序结果验证；运行时侧通过分块、地址规划、指令生成和指令缓存降低重复调度成本，并通过不同缓存状态的完整推理测试验证；通过将轻量级推理框架中的核心算子直接映射到NPU后端执行，该机制既降低了模型变化时的适配成本，又保留了框架入口和软件回退能力，更适合端侧轻量模型、多模型快速部署和NPU架构验证。

3.3特性成果

3.3.1缓存状态与推理性能

本版以MNIST与KWS的完整TinyMaix推理调用比较缓存状态。MNIST首次、指令流未命中、热缓存耗时分别为5.19508、2.29884、1.95034 ms；KWS分别为38.64252、31.79060、28.06528 ms。MNIST后两档复用权重、配置和量化计划，KWS没有持久权重复用。未命中状态不代表所有缓存均冷启动；每轮重新输入后，CPU与NPU最终分数及中间层输出均进行一致性检查。

XC7A100T 50 MHz完整TinyMaix推理耗时

任务	首次NPU / ms	指令流未命中 / ms	热缓存 / ms
MNIST	5.19508	2.29884	1.95034
KWS	38.64252	31.79060	28.06528

图3-8 XC7A100T平台缓存状态与推理耗时

3.3.2 端到端推理性能

表3-1典型网络端到端推理性能对比

网络模型	主控CPU	推理框架	CPU推理/ms	加速后/ms	加速比
MNIST CNN	VexRiscv	TinyMaix	22.96778	1.95034	11.78×
KWS CNN	VexRiscv	TinyMaix	275.87612	28.06528	9.83×

最终演示选用XC7A100T，VexRiscv与NPU均为50 MHz。CPU/NPU使用同一TinyMaix模型和输入，取5次中位数；加速后耗时采用热缓存结果，计时覆盖完整推理调用，不含预处理与串口传输。MNIST首次NPU为5.19508 ms，KWS首次为38.64252 ms。

端到端网络测试用于验证轻量化算子映射与异构协同执行机制。MNIST与语音关键词识别在最终XC7A100T演示平台上分别取得11.78倍、9.83倍热缓存加速。

第四部分 总结

4.1 可扩展之处

后续工作可围绕三项机制继续完善：在细粒度流水化NPU加速结构方面，扩展阵列规模、片上缓存和多通道DMA；在算子运行时调度与轻量级指令流缓存机制方面，支持更多动态形状和更细粒度的指令复用策略；在轻量化算子映射与异构协同执行机制方面，扩展ReLU6、池化、归一化等算子支持，并针对深度卷积和小矩阵尾块优化映射策略。

4.2 心得体会

本文面向轻量级神经网络推理框架设计并实现了一套CPU-NPU异构硬件加速系统。作品围绕轻量化部署机制、算子运行时调度与指令流缓存机制、细粒度流水化NPU加速结构，形成了从问题分析、方案论证、软件运行时、硬件流水到系统测试的完整技术闭环。

与只针对单一网络的硬连线加速器相比，本作品更加关注三条技术链路的边界清晰与测试闭环。硬件侧通过DMA、脉动阵列、结果重排和PPU后处理流水提供细粒度流水化NPU加速结构，并通过模块仿真、资源和时序结果验证；运行时侧通过分块、地址规划、指令生成和指令缓存降低重复调度成本，并通过不同缓存状态的完整推理测试验证；通过将轻量级推理框架中的核心算子直接映射到NPU后端执行，该机制既降低了模型变化时的适配成本，又保留了框架入口和软件回退能力，更适合端侧轻量模型、多模型快速部署和NPU架构验证。

本设计面向轻量级神经网络在端侧场景下的快速部署需求。系统保留TinyMaix轻量级推理框架入口，在运行时完成算子映射、数据分块、地址规划和硬件指令生成，并通过指令流缓存机制减少重复调度开销。因此，本设计在保持较好推理加速效果的同时，兼顾了多网络适配、运行时指令复用和轻量化部署能力。

第五部分 参考文献

[1] Mohd Noor M H, Ige A O. A survey on state-of-the-art deep learning applications and challenges[J]. Engineering Applications of Artificial Intelligence, 2025, 159: 111225.

[2] Mienye I D, Swart T G. A comprehensive review of deep learning: Architectures, recent advances, and applications[J]. Information, 2024, 15(12): 755.

[3] 施巍松, 孙辉, 曹杰, 等. 边缘计算: 万物互联时代新型计算模型[J]. 计算机研究与发展, 2017, 54(5): 907-924.

[4] Deng S, Zhao H, Fang W, et al. Edge intelligence: The confluence of edge computing and artificial intelligence[J]. IEEE Internet of Things Journal, 2020, 7(10): 7779-7807.

[5] Chen J, Ran X. Deep learning with edge computing: A review[J]. Proceedings of the IEEE, 2019, 107(8): 1655-1674.

[6] Sze V, Chen Y H, Yang T J, et al. Efficient processing of deep neural networks: A tutorial and survey[J]. Proceedings of the IEEE, 2017, 105(12): 2295-2329.

[7] Howard A G, Zhu M, Chen B, et al. MobileNets: Efficient convolutional neural networks for mobile vision applications[J]. arXiv preprint arXiv:1704.04861, 2017.

[8] Zhang X, Zhou X, Lin M, et al. ShuffleNet: An extremely computation-efficient convolutional neural network for mobile devices[C]. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018: 6848-6856.

[9] Han S, Mao H, Dally W J. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding[J]. arXiv preprint arXiv:1510.00149, 2015.

[10] Jacob B, Kligys S, Chen B, et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference[C]. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018: 2704-2713.

[11] Hennessy J L, Patterson D A. A new golden age for computer architecture: Domain-specific hardware/software co-design, enhanced security, open instruction sets, and agile chip development[J]. Computer, 2019, 52(2): 48-60.

[12] Jouppi N P, Young C, Patil N, et al. In-datacenter performance analysis of a tensor processing unit[C]. Proceedings of the 44th Annual International Symposium on Computer Architecture, 2017: 1-12.

[13]TensorFlow Developers. TensorFlow Lite[EB/OL]. https://tensorflow.google.cn/lite.

[14] Sipeed. TinyMaix[EB/OL]. https://github.com/sipeed/TinyMaix.

[15] 孟庆昊, 边丽蘅. 基于FPGA的神经网络硬件加速器研究综述[J]. 信号处理, 2025, 41(12): 1855-1873.

[16] Mohaidat T, Khalil K. A survey on neural network hardware accelerators[J]. IEEE Transactions on Artificial Intelligence, 2024, 5(8): 3801-3822.

[17] Deng L, Li G, Han S, et al. Model compression and hardware acceleration for neural networks: A comprehensive survey[J]. Proceedings of the IEEE, 2020, 108(4): 485-532.

[18]基于FPGA的Mobilenet神经网络加速器设计2025.DOI:10.27005/d.cnki.gdzku.2025.002057